ATLAS

SYSTEM DESIGN DOCUMENT

ATLAS Store Management System

A Rust-based, JSON-backed CLI for inventory, customers, suppliers, orders, sales, payments and reporting — engineered to industry standard.

DOCUMENT ID	VERSION	STATUS	DATE	CLASSIFICATION
ATLAS-SMS-2026-001	**1.0**	**Issued for Engineering Build**	**September 22, 2026**	**Internal — Engineering**

Table of Contents

1. Overview
2. High-Level Architecture
3. Crate & Module Layout
4. Storage Layer Design
5. Domain Model & JSON Schemas
6. CLI Command Tree
7. Key Business Workflows
8. Access Control (RBAC)
9. Reporting Engine
10. Error Handling & Validation
11. Concurrency & Data Integrity
12. Testing Strategy
13. Requirements Traceability (82/82 requirements)
14. Future Extensions (v2+)
15. Alignment with Industry System-Design Practice
16. Data Protection & Backup

Appendix A: Data Dictionary

Appendix B: Document Control

1. Overview

What ATLAS is, who it is for, and the standard it is built to

ATLAS is a role-based, offline-first Store Management System delivered as a single Rust binary. All state is persisted as JSON documents on the local filesystem — no database server, no network dependency required to operate. The design targets small-to-mid-size retail businesses that need a reliable point-of-sale, inventory, and order management system without infrastructure overhead, while still following industry-standard software architecture practice: layered architecture, the repository pattern, role-based access control, an immutable audit trail, and crash-safe atomic writes.

1.1 Design Goals

- **Correctness over cleverness** — stock, sales, and payment totals must never drift.
- **Crash-safe persistence** — no JSON file is ever left half-written.
- **Separation of concerns** — CLI parsing, business rules, and storage are independent layers.
- **Extensibility** — JSON storage is swappable for SQLite/Postgres later without touching business logic.
- **Auditable** — every meaningful state change is attributable to a user and a timestamp.

1.2 Non-Goals (v1)

- Multi-node / concurrent-process access (single-process, single-machine in v1).
- Networked sync or a REST API (planned as a v2 extension — see Section 14).
- A graphical interface (CLI only).

> **Direction:** *ATLAS is scoped as a single-store, single-binary system in v1. The architecture below is deliberately layered so that multi-branch sync, a REST API, or a database swap can be added later without a rewrite of business logic.*

2. High-Level Architecture

How the system is layered, and why

ATLAS uses a four-layer architecture. Each layer depends only on the layer directly beneath it, which keeps business rules independent of how data happens to be stored.



Every service depends only on repository **traits**, never on a concrete file implementation, so storage can later be swapped for SQLite by writing one new repository implementation — with zero changes to service code.

3. Crate & Module Layout

How the Rust project is organised on disk

```
atlas/
├── Cargo.toml
├── data/                          # runtime JSON store (git-ignored)
│   ├── users.json
│   ├── products.json
│   ├── categories.json
│   ├── stock_movements.json
│   ├── suppliers.json
│   ├── purchases.json
│   ├── customers.json
│   ├── sales.json
│   ├── orders.json
│   ├── payments.json
│   ├── returns.json
│   ├── notifications.json
│   ├── audit_log.json
│   └── sessions.json
├── src/
│   ├── main.rs                    # entrypoint, wires everything up
│   ├── cli/                       # user_cmds, product_cmds, inventory_cmds,
│   │                               # supplier_cmds, customer_cmds, sale_cmds,
│   │                               # order_cmds, payment_cmds, return_cmds,
│   │                               # report_cmds, audit_cmds
│   ├── domain/                    # user, product, inventory, supplier,
│   │                               # customer, sale, order, payment,
│   │                               # return_, audit
│   ├── services/                  # auth_service, product_service,
│   │                               # inventory_service, supplier_service,
│   │                               # customer_service, sale_service,
│   │                               # order_service, payment_service,
│   │                               # return_service, report_service,
│   │                               # notification_service, audit_service
│   ├── repository/
│   │   ├── mod.rs                 # JsonRepository<T> generic engine
│   │   ├── traits.rs              # Repository<T> trait definitions
│   │   └── json_store.rs          # atomic read/write/lock primitives
│   ├── auth/
│   │   ├── rbac.rs                # roles & permission matrix
│   │   └── session.rs             # login session tokens
│   ├── errors.rs                  # AtlasError (thiserror)
│   └── utils/                     # ids, time, format
└── tests/
    ├── integration/
    └── fixtures/
```

4. Storage Layer Design

How data is safely persisted to JSON on disk

4.1 Principles

- **One JSON file = one collection** (table-equivalent). Each file holds a JSON array of records.
- **Atomic writes**: every write goes to a `*.tmp` file, is `fsync`'d, then atomically renamed over the target file. This guarantees the store is never left corrupted mid-write.
- **File locking**: `fs2::FileExt::lock_exclusive()` wraps every write, and `lock_shared()` wraps every read, guarding against two ATLAS processes running concurrently.
- **In-memory cache**: on startup, each repository loads its JSON file into a `Vec<T>` / `HashMap<Uuid, T>` index. All reads are served from memory; writes update memory then flush to disk.
- **Schema versioning**: every JSON file has a top-level envelope, `{ "schema_version": 1, "records": [...] }`, so future migrations can detect and upgrade old files.

4.2 Generic Repository Trait

```
pub trait Repository<T: Identifiable> {
    fn find_all(&self) -> Vec<T>;
    fn find_by_id(&self, id: Uuid) -> Option<T>;
    fn insert(&mut self, item: T) -> Result<T, AtlasError>;
    fn update(&mut self, item: T) -> Result<T, AtlasError>;
    fn delete(&mut self, id: Uuid) -> Result<(), AtlasError>;
    fn query<F: Fn(&T) -> bool>(&self, predicate: F) -> Vec<T>;
}
```

A single generic `JsonRepository<T>` implements this trait for any `T: Serialize + DeserializeOwned + Identifiable + Clone`. Entity-specific repositories (e.g. `ProductRepository`) wrap it and add typed convenience queries such as `find_low_stock()` or `find_by_sku()`.

4.3 Envelope Example — `products.json`

```
{
  "schema_version": 1,
  "records": [
    {
      "id": "b2e6b2b0-6b0a-4e2a-9b0a-0a2a2a2a2a2a",
      "sku": "ATL-0001",
      "name": "500ml Bottled Water",
      "category_id": "d1e2...",
      "price": 200.0,
      "active": true,
      "created_at": "2026-09-22T10:00:00Z",
      "updated_at": "2026-09-22T10:00:00Z"
    }
  ]
}
```

5. Domain Model & JSON Schemas

The core entities, their fields, and how they relate

All IDs are UUID v4. All timestamps are ISO-8601 UTC via `chrono`.

5.1 User & Auth

```
User {
  id, username, password_hash, full_name, role: Role, active,
  must_change_password: bool,   // true until the staff member completes registration
  created_at
}
Role = Admin | Manager | Cashier
Session { token, user_id, issued_at, expires_at }
```

`must_change_password` is what turns account creation into a real registration step: Admin provisions the account with a temporary password (Section 6, `atlas user create`), and the staff member's first `atlas auth login` forces an immediate `atlas auth set-password` before any other command is accepted — see Section 8.1.

5.2 Product & Category

```
Category { id, name, description }
Product {
  id, sku, name, description, category_id,
  price, cost_price, active, created_at, updated_at
}
```

5.3 Inventory

```
StockLevel { product_id, quantity_on_hand, reorder_threshold }
StockMovement {
  id, product_id, change: i64, reason: StockReason,
  reference_id (sale_id/order_id/purchase_id/None),
  performed_by, timestamp
}
StockReason = Restock | Sale | Adjustment | Return | Damage
```

Current quantity is a *materialized* value kept in step with the sum of `StockMovement.change` for a product — see Section 7.3 for how consistency is guaranteed.

5.4 Supplier & Purchases

```
Supplier { id, name, contact_person, phone, email, address, active }
Purchase {
  id, supplier_id, items: [PurchaseItem { product_id, quantity, unit_cost }],
  total_cost, purchased_at, recorded_by
}
```

5.5 Customer

```
Customer { id, name, phone, email, address, created_at }
```

Order history is queried rather than embedded, avoiding duplication — see `order_service::find_by_customer`.

5.6 Sale (over-the-counter, immediate)

```
Sale {
  id, cashier_id, customer_id: Option<Uuid>,
  items: [SaleItem { product_id, quantity, unit_price, subtotal }],
  total, payment_id, created_at
}
```

5.7 Order (may be created ahead of fulfilment/payment)

```
Order {
  id, customer_id, created_by,
  items: [OrderItem { product_id, quantity, unit_price }],
  total, status: OrderStatus, created_at, updated_at
}
OrderStatus = Pending | Confirmed | Fulfilled | Cancelled
```

5.8 Payment

```
Payment {
  id, reference_type: SaleOrOrder, reference_id,
  amount, method: PaymentMethod, status: PaymentStatus,
  processed_by, created_at
}
PaymentMethod = Cash | Card | BankTransfer | MobileMoney
PaymentStatus = Pending | Success | Failed | Refunded
```

5.9 Return

```
ReturnRecord {
  id, sale_id, product_id, quantity, reason,
  refund_amount: Option<f64>, accepted_by, created_at
}
```

5.10 Notification

```
Notification {
  id, kind: NotificationKind, message,
  target_role: Option<Role>, read, created_at
}
NotificationKind = LowStock | OutOfStock | ProductUnavailable | NewOrder
                 | PaymentSuccess | PaymentFailed | OrderCancelled
```

`OutOfStock` fires when `quantity_on_hand` reaches zero; `ProductUnavailable` fires separately when a product is explicitly deactivated (`atlas product deactivate`) — the two are distinct triggers because a product can become unavailable by an administrative decision even while stock remains.

Since ATLAS is a CLI with no persistent daemon, notifications are generated *synchronously* at the moment a triggering event occurs — e.g. a sale drops stock below its threshold — written to `notifications.json`, and surfaced two ways: (a) printed immediately as a highlighted terminal banner, and (b) listed on demand via `atlas notify list`.

5.11 Audit Log

```
AuditEntry {
  id, user_id, action: String,       // e.g. "product.create", "sale.void"
  entity_type, entity_id,
  metadata: serde_json::Value,       // free-form diff/context
  timestamp
}
```

Every service method that mutates state calls `audit_service::record(...)` as its final step — see Section 7.1.

6. CLI Command Tree

Built with `clap` derive macros — one subcommand group per noun

```
atlas auth login --username <u>
atlas auth logout
atlas auth reset-password --username <u>
atlas auth set-password                     # forced on first login (must_change_password)

atlas user create   --username --full-name --role
atlas user list
atlas user set-role --id <id> --role <role>
atlas user deactivate --id <id>

atlas product add        --sku --name --category --price [--cost]
atlas product edit       --id --field=value...
atlas product deactivate --id                # soft delete — sets active=false, fires ProductUnavailable
atlas product remove     --id                # hard delete — only if never referenced by a sale/order/movement
atlas product show       --id | --sku
atlas product search     --query <text>
atlas category add --name
atlas category list

atlas inventory show --product-id
atlas inventory restock  --product-id --quantity --supplier-id
atlas inventory adjust   --product-id --quantity --reason
atlas inventory low-stock
atlas inventory out-of-stock
atlas inventory history --product-id

atlas supplier add / edit / deactivate / list
atlas supplier products --id <supplier-id>
atlas purchase record --supplier-id --items <sku:qty:cost,...>
atlas purchase list --supplier-id

atlas customer register / edit / search
atlas customer orders --id <customer-id>

atlas sale start                            # interactive line-item entry
atlas sale add-item   --sale-id --product-id --qty
atlas sale checkout   --sale-id --payment-method
atlas sale list --from <date> --to <date>
atlas sale show --id

atlas order create --customer-id --items <sku:qty,...>
atlas order add-item --order-id --product-id --qty
atlas order status --id --set <status>
atlas order cancel --id
atlas order show --id
atlas order list --customer-id | --status

atlas payment take --reference-id --amount --method
atlas payment refund --payment-id --amount
atlas payment show --id
atlas payment history --reference-id

atlas return record --sale-id --product-id --qty --reason
atlas return list

atlas report sales --period daily|weekly|monthly [--date]
atlas report product-sales --product-id
atlas report inventory
atlas report low-stock
atlas report purchases --supplier-id
atlas report payments
atlas report customer-orders --customer-id

atlas notify list [--unread]
atlas audit log --entity-type --entity-id --from --to

atlas backup export  --output <path> [--encrypt]
atlas backup restore --input <path> [--decrypt]
```

Global flags available on every command: `--format table|json`, `--data-dir <path>` (overrides the default `./data`), `--verbose`.

7. Key Business Workflows

How a mutation moves through the system, end to end

7.1 Standard Mutation Pattern

Every write command in ATLAS follows the same sequence:

```
1. CLI parses args → builds a command DTO
2. Service layer:
   a. Load current session → resolve acting user + role
   b. rbac::authorize(role, action) → Err(Unauthorized) if not permitted
   c. Validate input (quantities > 0, prices >= 0, etc.)
   d. Perform the mutation via repository
   e. audit_service::record(...)
   f. notification_service::maybe_notify(...) if a trigger condition is met
3. CLI renders success/failure to terminal
```

7.2 Sale Checkout Flow

```
atlas sale checkout --sale-id S --payment-method cash
  → sale_service::checkout(S, method)
      1. Load Sale S (must be "open" with ≥1 item)
      2. For each SaleItem: inventory_service::has_sufficient_stock?
         - if any insufficient → abort with ValidationError, no side effects
      3. Compute total (sum of subtotals)
      4. payment_service::create(Sale, total, method)
            → Payment { status: Success | Failed }
      5. If Payment succeeded:
           - inventory_service::deduct_stock(item) for each item
             → writes StockMovement{reason: Sale} + updates StockLevel
           - sale_service::mark_completed(S)
           - audit: "sale.checkout"
           - notification: check post-deduction stock →
             LowStock / OutOfStock if crossed threshold
      6. If Payment failed:
           - sale remains open, notification: PaymentFailed
      7. Print receipt (table) to terminal
```

This sequencing guarantees stock is **never** deducted before payment succeeds, and a failed payment never leaves a partially-updated inventory.

7.3 Stock Consistency Rule

`StockLevel.quantity_on_hand` is a materialized value updated transactionally with every `StockMovement` insert. Both writes go through `inventory_service::apply_movement()`, which writes the movement record and updates the level in a single locked critical section spanning both `stock_movements.json` and `stock_levels.json` — avoiding a recomputed running sum on every read while still keeping full movement history for audit and reporting.

7.4 Return Flow

```
atlas return record --sale-id S --product-id P --qty Q --reason "damaged"
  1. Validate S exists, is completed, and contains P with qty >= Q
  2. Create ReturnRecord
  3. inventory_service::apply_movement(P, +Q, reason=Return)
     [only if the item is accepted back into resale stock]
  4. If refund applicable → payment_service::refund(original_payment, amount)
  5. audit: "return.record"
```

7.5 Order → Payment → Fulfilment

Orders are deliberately decoupled from immediate stock deduction: stock is only deducted when an order transitions to `Fulfilled`, reusing `inventory_service::deduct_stock`. This mirrors the sale flow while allowing a `Pending` / `Confirmed` window — for pre-orders or delivery orders — without holding stock hostage against an order that may still be cancelled.

8. Access Control (RBAC)

What each role can and cannot do

Capability	Admin	Manager	Cashier
Manage staff accounts & roles	✅	❌	❌
Product CRUD	✅	✅	❌
Supplier CRUD / purchases	✅	✅	❌
Inventory restock / adjust	✅	✅	❌
View inventory & low-stock	✅	✅	✅
Create sale / take payment	✅	✅	✅
Process refunds	✅	✅	❌
Manage customers	✅	✅	✅
Create / cancel orders	✅	✅	✅ (create only)
Record returns	✅	✅	❌
Manage backups	✅	❌	❌
View all reports	✅	✅	Own sales only
View audit log	✅	✅	❌

Implementation: `rbac::authorize(role: Role, action: Action) -> Result<(), AtlasError>`, where `Action` is a fine-grained enum (`Action::ProductCreate`, `Action::RefundProcess`, ...) checked against a static permission matrix — no string comparisons, so the compiler catches typos in permission checks.

8.1 Registration & Authentication

- **Registration**: an Admin provisions a staff account with `atlas user create`, which sets a one-time temporary password and `must_change_password = true`. The staff member's first `atlas auth login` succeeds against the temporary password but every other command is blocked until they run `atlas auth set-password` — this two-step provision-then-set-your-own-password sequence is what satisfies "the system shall allow users to register and log in" for a staff-managed tool where accounts are not self-service.
- Passwords are hashed with **Argon2id** (the `argon2` crate) — never stored or logged in plaintext.
- `atlas auth login` verifies the hash, then issues a short-lived local session token stored in `sessions.json` (mirrored to `~/.atlas/session`) so subsequent commands in the same shell don't require re-login.
- `atlas auth reset-password` requires either the current password or Admin privilege.

9. Reporting Engine

How every report in Section 10 of the requirements is produced

`report_service` builds reports by composing repository queries — there is no separate reporting database. Each report type is a pure function over the in-memory collections:

- **Sales reports (daily/weekly/monthly)** — filter `sales.json` by date range, group by day/week/month, sum totals.
- **Product sales report** — aggregate `SaleItem`s by `product_id`.
- **Inventory report** — join `products.json` with `stock_levels.json`.
- **Low-stock / out-of-stock** — filter where `quantity_on_hand <= reorder_threshold` / `== 0`.
- **Purchase report** — filter `purchases.json` by supplier/date.
- **Payment report** — aggregate `payments.json` by method/status/date.
- **Customer order report** — filter `orders.json` by `customer_id`.

Output renders via `comfy-table` for terminal display, with a `--format json` option for machine-readable output that can be piped into other tools.

10. Error Handling & Validation

How ATLAS fails safely, and communicates why

- A central `AtlasError` enum (via `thiserror`) with variants: `NotFound`, `ValidationError(String)`, `Unauthorized`, `InsufficientStock`, `IoError`, `SerializationError`, `Conflict`.
- The domain layer performs structural validation — non-empty names, price ≥ 0, quantity > 0 — before any repository call, so invalid data never reaches disk.
- The CLI layer maps every `AtlasError` to a clear, human-readable message with a non-zero exit code — satisfying the requirement to "provide clear error messages when an operation fails" without leaking internals (stack traces are shown only with `--verbose`).

11. Concurrency & Data Integrity

What keeps the JSON store trustworthy under real-world use

- Single-process is the v1 assumption, but the `fs2` file-lock discipline means a second accidental invocation fails fast with a clear "store is locked" error instead of corrupting data.
- All multi-file mutations (e.g. sale checkout touching sales, stock, payments, and audit) acquire locks in a **fixed global lock order** — alphabetical by filename — to prevent deadlocks.
- The write path is always: mutate in-memory → serialize → write `.tmp` → `fsync` → `rename` → release lock. A crash at any point before `rename` leaves the original file untouched.

12. Testing Strategy

How correctness is verified at every layer

Layer	Test Type	Tooling
Domain	Unit tests on invariants (e.g. negative price rejected)	`#[cfg(test)]` in-module
Repository	Round-trip serialize/deserialize, atomic-write crash simulation	`tempfile` crate
Services	Integration tests per workflow (sale checkout, return, RBAC denial)	`tests/integration/` with fixture JSON stores
CLI	End-to-end invocation tests asserting stdout/exit code	`assert_cmd` + `predicates`

13. Requirements Traceability

Every requirement in the original brief, mapped to the design element that satisfies it — 82 of 82 met

This section replaces a "sample" mapping with a complete, line-by-line trace against the original *Store Management System — Requirements* document, grouped in the same 14 categories as the source. Every bullet from that document appears below exactly once.

13.1 User Management

Requirement	Design Element	Status
Allow users to register and log in	`atlas user create` (Admin provisions) + `atlas auth set-password` on first login + `atlas auth login` — Section 8.1	Met
Allow users to log out	`atlas auth logout` — Section 6	Met
Allow users to reset their passwords	`atlas auth reset-password` — Section 6, 8.1	Met
Allow administrators to create and manage staff accounts	`atlas user create / list / deactivate` — Section 6	Met
Allow administrators to assign roles to staff	`atlas user set-role`, `Role` enum — Sections 5.1, 6	Met

13.2 Product Management

Requirement	Design Element	Status
Allow authorized staff to add products	`atlas product add`, RBAC: Admin/Manager — Sections 6, 8	Met
Allow authorized staff to edit products	`atlas product edit` — Section 6	Met
Allow authorized staff to remove or deactivate products	`atlas product deactivate` (soft) / `atlas product remove` (hard, unreferenced only) — Section 6	Met
Display product information	`atlas product show` — Section 6	Met
Allow users to search for products	`atlas product search` — Section 6	Met
Allow products to be grouped into categories	`Category` entity, `Product.category_id` — Sections 5.2, 6	Met
Allow staff to set product prices	`Product.price`, `atlas product edit --field price=...` — Sections 5.2, 6	Met

13.3 Inventory Management

Requirement	Design Element	Status
Display the available quantity of each product	`StockLevel.quantity_on_hand`, `atlas inventory show` — Sections 5.3, 6	Met
Allow staff to add new stock	`atlas inventory restock` — Section 6	Met
Allow staff to reduce or adjust stock	`atlas inventory adjust` — Section 6	Met
Record every stock change	`StockMovement` — Sections 5.3, 7.3	Met
Identify products with low stock	`atlas inventory low-stock` — Sections 6, 9	Met
Identify products that are out of stock	`atlas inventory out-of-stock` — Sections 6, 9	Met
Allow staff to view stock history	`atlas inventory history` — Section 6	Met

13.4 Supplier Management

Requirement	Design Element	Status
Allow staff to add suppliers	`atlas supplier add` — Section 6	Met
Allow staff to update supplier information	`atlas supplier edit` — Section 6	Met
Allow staff to deactivate suppliers	`atlas supplier deactivate` — Section 6	Met
Store supplier contact information	`Supplier{contact_person, phone, email, address}` — Section 5.4	Met
Allow staff to view products supplied by each supplier	`atlas supplier products --id` — Section 6	Met

Keep a record of purchases from suppliers	`Purchase` entity, `atlas purchase record/list` — Sections 5.4, 6	Met

13.5 Customer Management

Requirement	Design Element	Status
Allow staff to register customers	`atlas customer register` — Section 6	Met
Allow staff to update customer information	`atlas customer edit` — Section 6	Met
Allow staff to search for customers	`atlas customer search` — Section 6	Met
Keep customer order history	`atlas customer orders`, `order_service::find_by_customer` — Sections 5.5, 6	Met

13.6 Sales Management

Requirement	Design Element	Status
Allow staff to create a sale	`atlas sale start` — Section 6	Met
Allow staff to add multiple products to a sale	`atlas sale add-item` (repeatable) — Section 6	Met
Calculate the total cost of a sale	Checkout total computation — Section 7.2, step 3	Met
Reduce product stock after a successful sale	Section 7.2, step 5	Met
Record completed sales	`Sale` entity, `sale_service::mark_completed` — Sections 5.6, 7.2	Met
Allow staff to view previous sales	`atlas sale list / show` — Section 6	Met

13.7 Order Management

Requirement	Design Element	Status
Allow customers or staff to create orders	`atlas order create`, staff-mediated on the customer's behalf — Section 6	Met
Allow products to be added to an order	`atlas order add-item` — Section 6	Met
Calculate the order total	`Order.total`, computed from `OrderItem`s — Section 5.7	Met
Allow users to view order details	`atlas order show` — Section 6	Met
Allow authorized users to cancel orders	`atlas order cancel`, RBAC-gated — Sections 6, 8	Met
Track order status	`OrderStatus` enum — Section 5.7	Met
Keep order history	`atlas order list` — Section 6	Met

13.8 Payment Management

Requirement	Design Element	Status
Allow customers to make payments	`atlas payment take`, staff-processed at point of sale — Section 6	Met
Support different payment methods	`PaymentMethod` enum — Section 5.8	Met
Record payment details	`Payment` entity — Section 5.8	Met
Show the payment status	`PaymentStatus`, `atlas payment show` — Sections 5.8, 6	Met
Allow authorized staff to process refunds	`atlas payment refund`, RBAC: Admin/Manager — Sections 6, 8	Met
Keep payment history	`atlas payment history` — Section 6	Met

13.9 Returns

Requirement	Design Element	Status
Allow authorized staff to record returned products	`atlas return record`, RBAC: Admin/Manager — Sections 6, 7.4, 8	Met
Record the reason for a return	`ReturnRecord.reason` — Section 5.9	Met
Update inventory when a returned product is accepted	Section 7.4, step 3	Met

Record refunds where applicable	Section 7.4, step 4 (`payment_service::refund`)	Met

13.10 Reports

Requirement	Design Element	Status
Daily sales reports	`atlas report sales --period daily` — Sections 6, 9	Met
Weekly sales reports	`atlas report sales --period weekly` — Sections 6, 9	Met
Monthly sales reports	`atlas report sales --period monthly` — Sections 6, 9	Met
Product sales reports	`atlas report product-sales` — Sections 6, 9	Met
Inventory reports	`atlas report inventory` — Sections 6, 9	Met
Low-stock reports	`atlas report low-stock` — Sections 6, 9	Met
Purchase reports	`atlas report purchases` — Sections 6, 9	Met
Payment reports	`atlas report payments` — Sections 6, 9	Met
Customer order reports	`atlas report customer-orders` — Sections 6, 9	Met

13.11 Notifications

Requirement	Design Element	Status
A product reaches low stock	`NotificationKind::LowStock` — Section 5.10	Met
A product becomes unavailable	`NotificationKind::ProductUnavailable` (deactivation trigger) — Section 5.10	Met
A new order is created	`NotificationKind::NewOrder` — Section 5.10	Met
A payment is successful	`NotificationKind::PaymentSuccess` — Section 5.10	Met
A payment fails	`NotificationKind::PaymentFailed` — Section 5.10	Met
An order is cancelled	`NotificationKind::OrderCancelled` — Section 5.10	Met

13.12 Access Control

Requirement	Design Element	Status
Restrict features based on user roles	RBAC matrix, `rbac::authorize` — Section 8	Met
Administrators have access to management functions	RBAC matrix, Admin column — Section 8	Met
Managers have access to store management functions	RBAC matrix, Manager column — Section 8	Met
Cashiers have access to sales and payment functions	RBAC matrix, Cashier column — Section 8	Met
Staff only access information they are authorized to access	`rbac::authorize` enforced on every mutation and read path — Sections 7.1, 8	Met

13.13 Audit

Requirement	Design Element	Status
Record important user actions	`AuditEntry`, written at the end of every mutation — Sections 5.11, 7.1	Met
Record who performed an action	`AuditEntry.user_id` — Section 5.11	Met
Record when the action happened	`AuditEntry.timestamp` — Section 5.11	Met
Allow authorized users to view activity history	`atlas audit log`, RBAC: Admin/Manager — Sections 6, 8	Met

13.14 System Requirements

Requirement	Design Element	Status
Available to authorized users	Session-gated CLI, `atlas auth login` — Section 8.1	Met

Protect user and business information	Argon2id hashing, file-permission hardening, encrypted backups — Sections 8.1, 16.1, 16.2	Met
Validate user input	Domain-layer structural validation before any write — Section 10	Met
Prevent unauthorized access	RBAC + session authentication — Section 8	Met
Maintain accurate product, inventory, order, and payment records	Atomic writes, stock consistency rule, fixed lock ordering — Sections 4.1, 7.3, 11	Met
Provide clear error messages when an operation fails	`AtlasError` → human-readable CLI mapping — Section 10	Met

14. Future Extensions (v2+)

Where ATLAS goes after v1, without a rewrite

- Swap `JsonRepository<T>` for a SQLite-backed implementation (same `Repository<T>` trait) once concurrent/multi-terminal access is needed.
- Add a thin REST API layer (`axum`) reusing the existing service layer unchanged, enabling a web or mobile front-end.
- A background sync daemon for multi-branch stores.
- Export reports to CSV/PDF.

> **Direction:** *Every layer boundary in this design — CLI, service, domain, repository — exists specifically so that these extensions are additive. No future item on this list requires touching the domain layer's business rules.*

15. Alignment with Industry System-Design Practice

Checking ATLAS against the standard patterns, techniques and vocabulary used across the industry

Industry system-design practice (scaling, replication, distributed-transaction patterns, CQRS, sharding, message queues, and formal artefacts like data dictionaries and interface contracts) is written primarily for networked, multi-node systems. ATLAS v1 is a deliberately single-node, offline-first CLI, so the table below states, pattern by pattern, whether it applies now, and — where it does not — how the layered architecture in Sections 2–4 keeps the door open for it in v2 without a rewrite.

Pattern / Concept	Applies to ATLAS v1?	How it is addressed
Horizontal scaling	Not applicable	Single-node by design (Section 1.2). The v2 branch-sync roadmap (Section 14) is the scaling path, not more nodes today.
Vertical scaling	Trivial	A native Rust binary uses whatever CPU/RAM the host provides with no code changes.
Redundancy & replication	Partial	Atomic write + `fsync` + rename (Section 4.1) gives durability at the point of write. Multi-copy replication is a v2 concern once branch sync exists.
CAP theorem	Not applicable yet	A single node has no network partition to tolerate. It becomes directly relevant the moment v2 adds multi-branch sync — see the note below.
Microservices	Deliberately not adopted	ATLAS ships as one binary. The service/repository layer boundary (Section 2) preserves the option to extract a service later without touching business rules.
Message queues	Not needed at this scale	In-process function calls are sufficient for one user, one process. The audit log (Section 5.11) already records every event in order, so it can seed a real queue later if needed.
File systems	Core dependency	The local filesystem is the system of record. Section 4.1's atomic-write discipline is the standard technique for keeping a flat-file store crash-safe.
Two-Phase Commit (2PC)	Not used	Single-writer, single-node — there is nothing to coordinate a vote across. If v2 needs cross-branch consistency, a Saga-style approach (below) is preferred over 2PC, which does not tolerate the flaky connectivity ATLAS's users have.
CQRS	Present in a lightweight form	Reads are served from the in-memory repository cache (Section 4.1); writes go through validated command DTOs and the mutation pipeline (Section 7.1). This is CQRS's read/write separation without a second data store.
Saga pattern	Present in spirit	The return/refund flow (7.4) and the sale→stock→payment sequencing (7.2) already use ordered local steps with an explicit "undo" path (refund, no stock deduction on payment failure) rather than one all-or-nothing transaction — the same idea a distributed Saga formalizes across services.
Sharding	Not needed	A single store's data volume fits comfortably in memory and on disk; sharding only becomes relevant at multi-branch/multi-tenant scale.
Proxy servers / load balancing	Not applicable	ATLAS exposes no network service in v1 — there is nothing to sit in front of.

> **On CAP, ahead of v2:** *the moment branch-to-branch sync (Section 14) ships, ATLAS becomes a distributed system and CAP applies directly. The recommended default is to favour* **availability and partition tolerance** *over strict consistency — a branch must keep selling while offline — and reconcile conflicting writes when connectivity returns, using the same compensating-action (Saga) logic already proven in the return flow.*

15.1 Interface Contract

The CLI itself is ATLAS's API contract, in the same sense the industry uses "APIs and contracts" for networked systems: every command in Section 6 has a fixed set of flags (its request shape) and a fixed `--format table|json` output (its response shape). Because the JSON output shape is part of the contract, it must be covered by the CLI integration tests in Section 12 exactly like a REST response body would be — a breaking field rename is a contract break, not just a formatting change.

15.2 Documentation Artefacts Carried by This Document

- **Architecture diagram** — Section 2.
- **Data dictionary** — Appendix A, formalizing the field-level detail introduced in Section 5.
- **Decision table** — the RBAC matrix in Section 8 is a decision table: role × action → permitted/denied.
- **Pseudocode** — the workflow listings in Section 7 (checkout, return) are the pseudocode layer between the prose requirement and the

Rust implementation.

16. Data Protection & Backup

How ATLAS protects user and business information at rest, and recovers from loss

16.1 Data-at-Rest Protections

- The `data/` directory and every file inside it are created with restrictive permissions (`0600` for files, `0700` for the directory) on first run, so only the OS user running ATLAS can read the store.
- Password hashes use Argon2id (Section 8.1); the plaintext password is never written to disk, to `audit_log.json`, or to any log output.
- `--verbose` error output (Section 10) never includes field values from `User` or `Payment` records — only entity IDs and error codes — so a shared terminal transcript can't leak customer or payment data.
- Session tokens (`sessions.json`) expire after a configurable idle period and are invalidated immediately by `atlas auth logout`.

16.2 Backup & Restore

`atlas backup export --output <path> [--encrypt]` bundles every JSON collection into a single archive with its schema versions intact. With `--encrypt`, the archive is sealed with a passphrase-derived key (Argon2id for key derivation, AES-256-GCM for the payload) before being written — this is the mechanism that lets a backup leave the machine (off-site storage, a USB drive) without exposing business or customer data. `atlas backup restore --input <path> [--decrypt]` reverses the process, validating schema versions before overwriting `data/`.

This is an Admin-only capability (Section 8) precisely because a restore is a full-store overwrite — the same class of operation as the delete/deactivate actions already gated by RBAC.

Appendix A: Data Dictionary

Field-level detail for the entities introduced in Section 5

Product

Field	Type	Constraints
id	UUID v4	Primary key, generated on insert
sku	String	Unique, non-empty
name	String	Non-empty
category_id	UUID v4	Foreign key → Category.id
price	f64	>= 0
cost_price	f64	>= 0
active	bool	Default true
created_at / updated_at	ISO-8601 UTC datetime	Set/refreshed by the service layer, never by the caller

StockMovement

Field	Type	Constraints
id	UUID v4	Primary key
product_id	UUID v4	Foreign key → Product.id
change	i64	Positive (restock/return) or negative (sale/damage); never zero
reason	enum StockReason	Restock | Sale | Adjustment | Return | Damage
reference_id	Option<UUID v4>	Points to the sale/order/purchase that caused the movement, if any
performed_by	UUID v4	Foreign key → User.id
timestamp	ISO-8601 UTC datetime	Set by the service layer

Sale

Field	Type	Constraints
id	UUID v4	Primary key
cashier_id	UUID v4	Foreign key → User.id
customer_id	Option<UUID v4>	Foreign key → Customer.id; optional for walk-in sales
items	Vec<SaleItem>	Non-empty at checkout
total	f64	Computed, never accepted from the caller
payment_id	Option<UUID v4>	Set once checkout succeeds
created_at	ISO-8601 UTC datetime	Set on sale creation

Order

Field	Type	Constraints
id	UUID v4	Primary key
customer_id	UUID v4	Foreign key → Customer.id, required
items	Vec<OrderItem>	Non-empty
total	f64	Computed from items
status	enum OrderStatus	Pending | Confirmed | Fulfilled | Cancelled
created_at / updated_at	ISO-8601 UTC datetime	updated_at refreshed on every status change

Payment

Field	Type	Constraints
id	UUID v4	Primary key
reference_type	enum	Sale | Order
reference_id	UUID v4	Foreign key → Sale.id or Order.id per reference_type
amount	f64	> 0
method	enum PaymentMethod	Cash | Card | BankTransfer | MobileMoney
status	enum PaymentStatus	Pending | Success | Failed | Refunded
processed_by	UUID v4	Foreign key → User.id

Appendix B: Document Control

FIELD	DETAIL
Document Title	ATLAS Store Management System — System Design Document
Document ID	ATLAS-SMS-2026-001
Version	1.0
Status	Issued for Engineering Build
Source Requirements	Store Management System — Requirements
Target Stack	Rust (stable), serde / serde_json, clap, argon2, fs2, chrono, uuid, comfy-table, tracing
Persistence Model	JSON-backed, file-per-collection, offline, single-node
Date Issued	September 22, 2026
Classification	Internal — Engineering